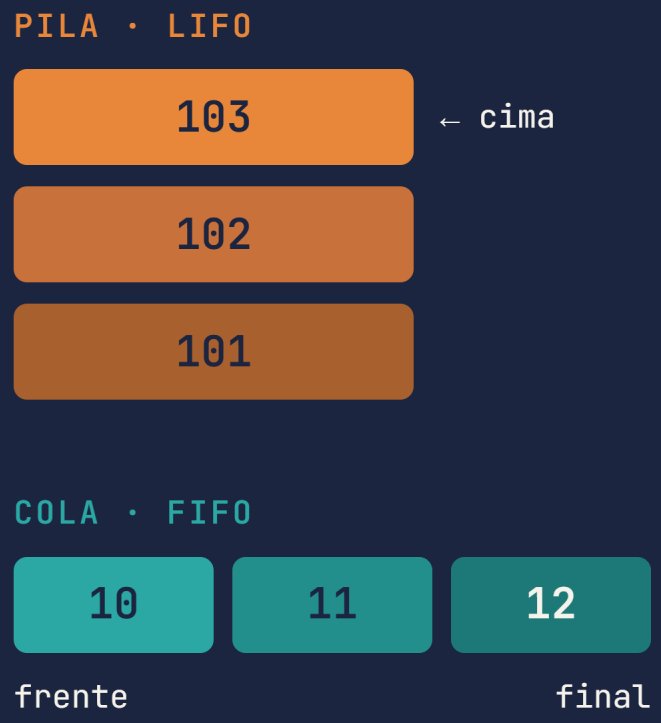
ESTRUCTURA DE DATOS · HUANCAYO · 2026

Sistema de gestión de procesos de un sistema operativo en C++

Lista enlazada, pila (LIFO) y cola (FIFO) hechas desde cero con punteros



Docente: Jaqueline Karol Inga Pérez

Rafael Zacarias, Jhovany Nico · Porras Benito, Flavio Isac

Buendia de la Rosa, Eric Daniel · Mayor Marticorena, Leonel Eduardo

EL PROYECTO

¿Qué queremos resolver?

Procesos

Se registran, se buscan y se ordenan por prioridad

`Lista enlazada`

Memoria RAM

Se asigna y se libera desde el bloque más reciente

`Pila · LIFO`

CPU

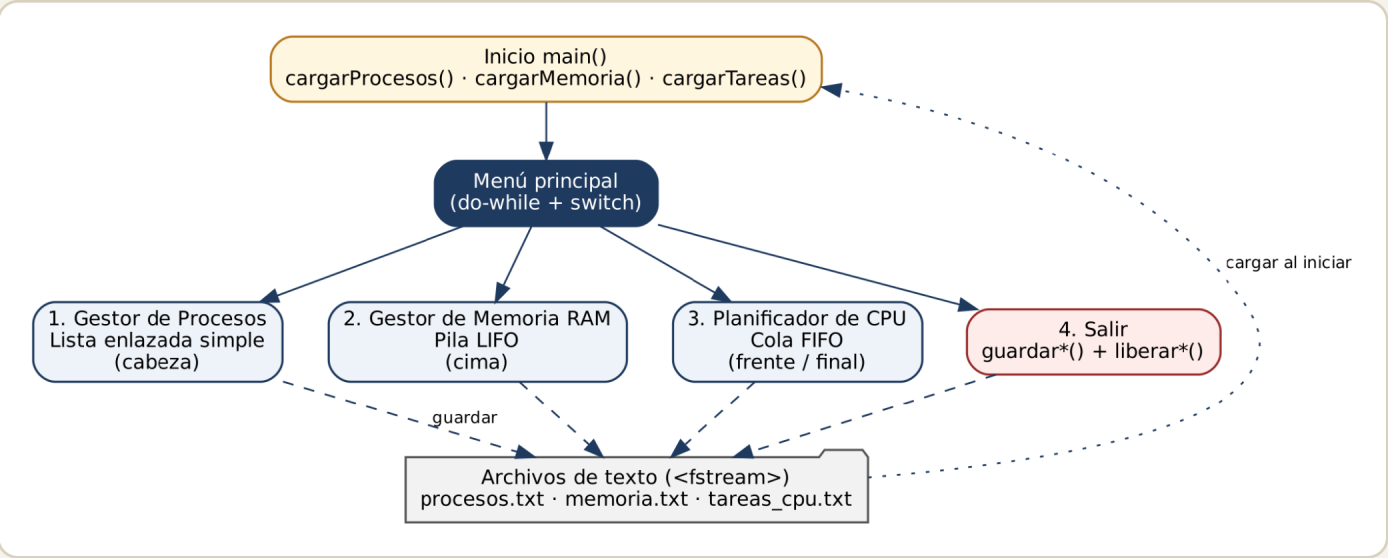
Las tareas se atienden en el orden en que llegan

`Cola · FIFO`

Un sistema operativo hace estas tres cosas a la vez, cada una con su propia lógica de acceso. Nuestro reto fue elegir la estructura lineal correcta para cada una e implementarla con punteros y **new / delete**, sin contenedores de la biblioteca estándar.

DISEÑO

Cómo está organizado el programa



01 · Al iniciar

main() carga procesos.txt, memoria.txt y tareas_cpu.txt.

02 · Menú principal

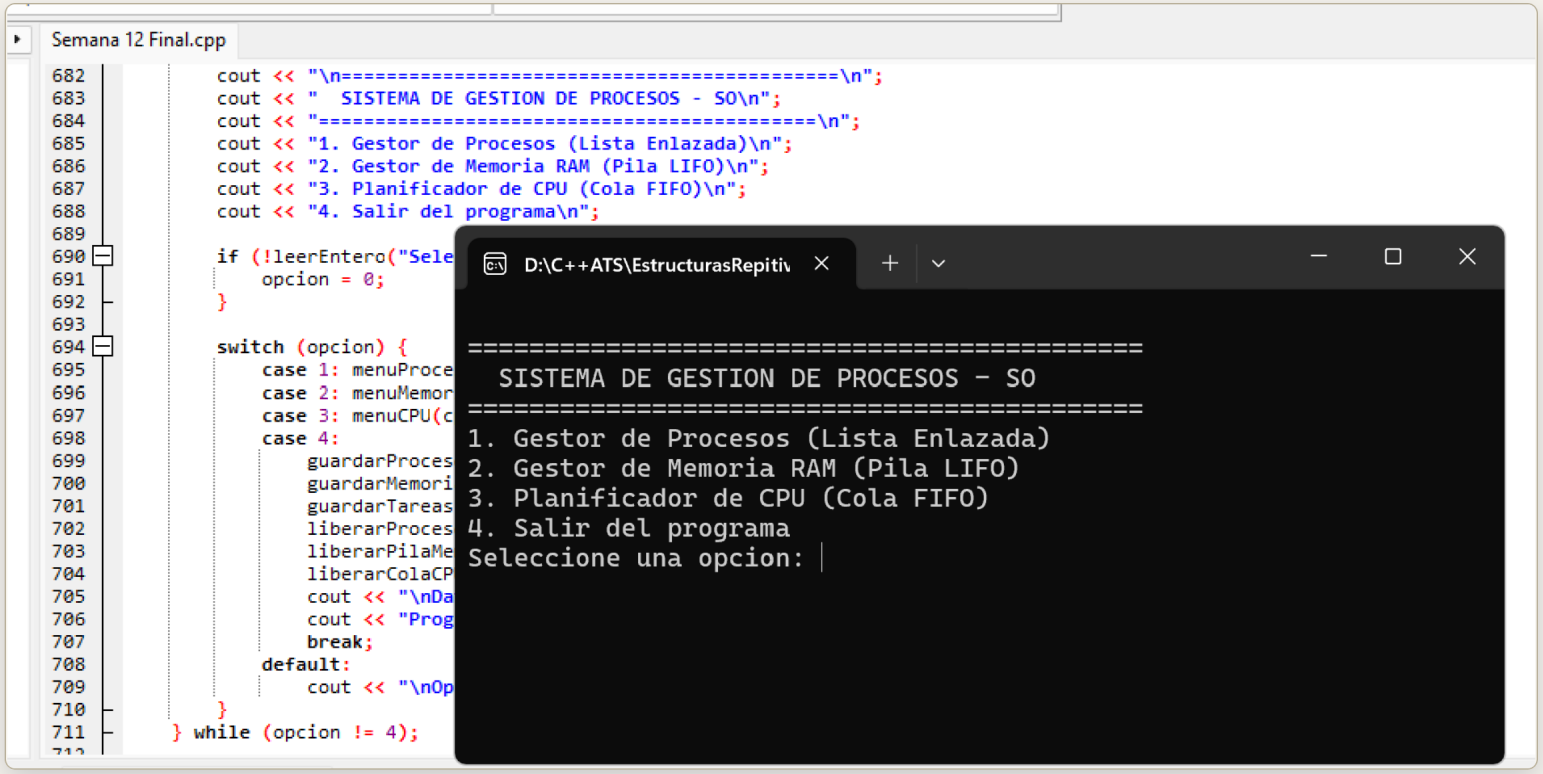
Un do-while repite el menú y un switch manda al módulo elegido.

03 · Al salir

Se guardan los tres archivos y se libera toda la memoria dinámica con delete.

CÓDIGO · MAIN()

El menú principal en funcionamiento



Captura: main() en Dev-C++ con la consola del programa ejecutándose

`do { … } while`
El menú se muestra al menos una vez y se repite hasta elegir 4.

`switch (opcion)`
Cada caso llama a su módulo: menuProcesos, menuMemoria o menuCPU.

`case 4`
Guarda los tres archivos y libera la memoria antes de terminar.

CÓDIGO · STRUCT

Registros y nodos

Proceso

```
int idProceso;
char nombre[30];
int prioridad;
```

NodoLista

`dato + siguiente`

BloqueMemoria

```
int idBloque;
int tamanoMB;
```

NodoPila

`dato + siguiente`

TareaCPU

```
int idTarea;
int idProceso;
char descripcion[50];
```

NodoCola

`dato + siguiente`

Separamos el **dato** (un registro completo) del **puntero siguiente**. Así el mismo registro sirve para leer, buscar y guardar en archivo sin depender de los enlaces. Están definidos entre las líneas 46 y 84.

PILA · LIFO

La pila: gestor de memoria RAM

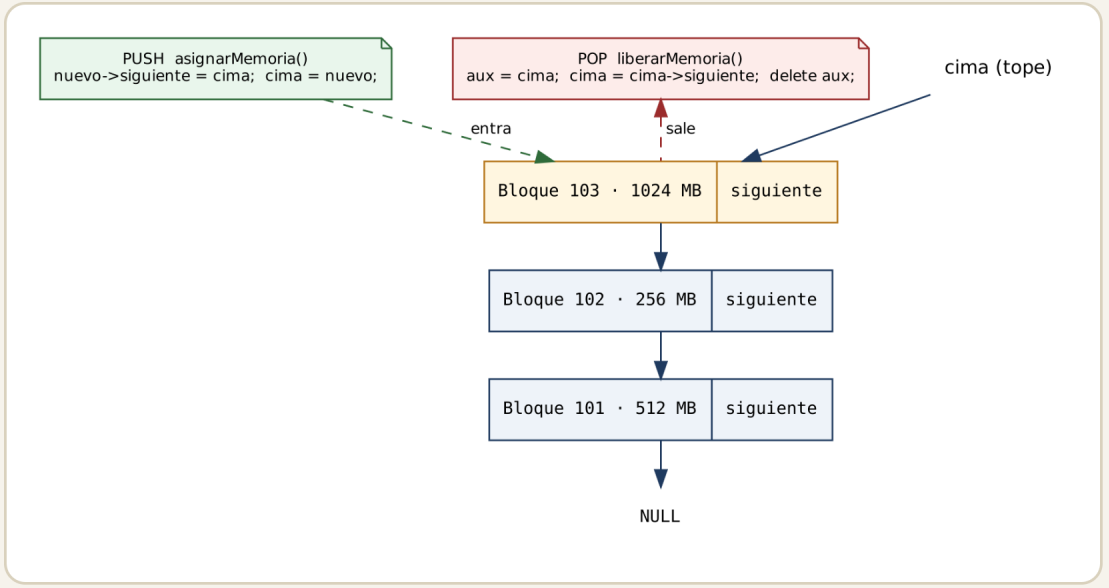
LIFO: Last In, First Out

El último que entra es el primero que sale, como una pila de platos: solo puedes tomar el de arriba.

`push`
apila: el bloque nuevo queda en la cima

`pop`
desapila: se retira el bloque de la cima

En el proyecto: la memoria RAM. El último bloque asignado es el primero en liberarse.



Pila después de tres push: el bloque 103 está en la cima.

CÓDIGO · PILA

Push y pop en C++

PUSH asignarMemoria() · líneas 369–372

```
368
369     NodoPila* nuevo = new NodoPila;
370     nuevo->dato = nuevoDato;
371     nuevo->siguiente = cima;
372     cima = nuevo;
373
```

`new NodoPila`
se crea el nodo en memoria dinámica

`nuevo→dato = nuevoDato;`
se guarda el ID y el tamaño del bloque

`nuevo→siguiente = cima;`
se enlaza encima de la pila actual

`cima = nuevo;`
ahora el bloque nuevo es el tope

POP liberarMemoria() · líneas 383–389

```
383     NodoPila* aux = cima;
384     cout << "\nBloque de memoria liberado del tope (Pop) -> ID: " << aux->dato.idBloque
385          << " | Tamano: " << aux->dato.tamanoMB << " MB\n";
386
387     cima = cima->siguiente;
388     delete aux;
389     guardarMemoria(cima);
390 }
```

`NodoPila* aux = cima;`
aux guarda el bloque del tope

`cout << … aux→dato …`
se muestra el bloque que se libera

`cima = cima→siguiente;`
la cima baja al bloque de abajo

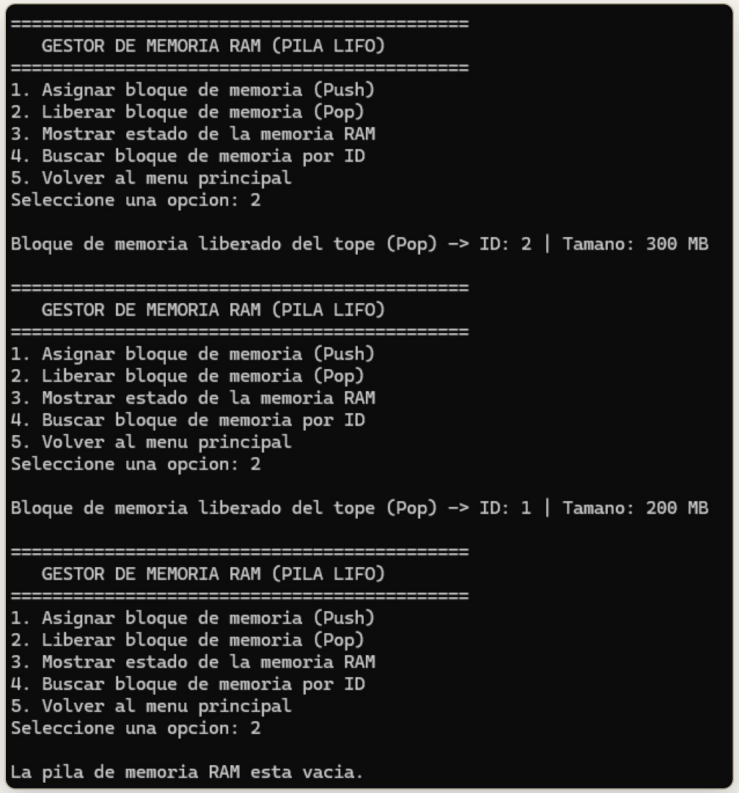
`delete aux;`
se libera la memoria del nodo

EJECUCIÓN · PILA

La pila funcionando



Captura: push y estado de la pila



Captura: pop sucesivos hasta vaciar

Push

El bloque 4 entró último y aparece marcado como [TOPE RAM].

Pop

Sale primero el último bloque: se libera el ID 2 (300 MB) y después el ID 1 (200 MB).

Pila vacía

Si no quedan bloques, el programa avisa y no accede a memoria liberada.

COLA · FIFO

La cola: planificador de CPU

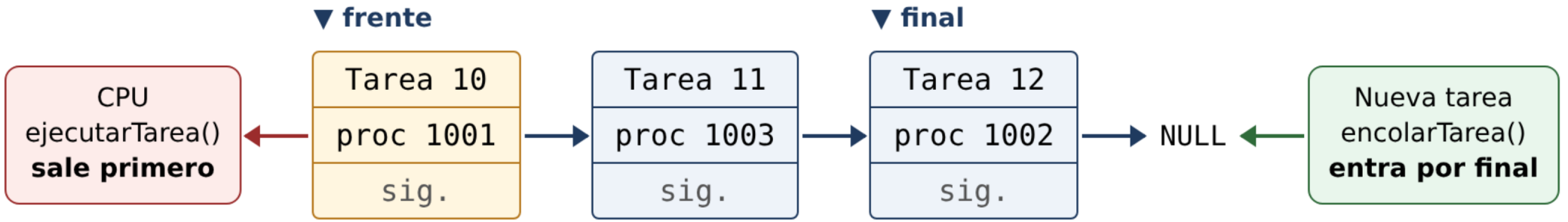
FIFO: First In, First Out. La primera tarea en llegar es la primera en ejecutarse, como la fila de un banco. Es la planificación First-Come, First-Served.

`encolar → por el final`

El nodo nuevo se enlaza después del último y **final** se actualiza.

`ejecutar → por el frente`

Se retira el nodo del **frente** y frente avanza al siguiente.



CÓDIGO · COLA

Encolar y ejecutar en C++

ENCOLAR encolarTarea() · líneas 468–477

```
468     NodoCola* nuevo = new NodoCola;
469     nuevo->dato = nuevoDato;
470     nuevo->siguiente = NULL;
471
472     if (cpu.frente == NULL) {
473         cpu.frente = nuevo;
474     } else {
475         cpu.final->siguiente = nuevo;
476     }
477     cpu.final = nuevo;
```

`nuevo→siguiente = NULL;`
el nodo nuevo será el último

`if (cpu.frente == NULL)`
si la cola estaba vacía, entra como frente

`cpu.final→siguiente = nuevo;`
si no, se enlaza detrás del último

`cpu.final = nuevo;`
final apunta al nodo nuevo

EJECUTAR ejecutarTarea() · líneas 494–498

```
494     cpu.frente = cpu.frente->siguiente;
495     if (cpu.frente == NULL) {
496         cpu.final = NULL;
497     }
498     delete aux;
```

`cpu.frente = cpu.frente→siguiente;`
el frente avanza a la siguiente tarea

`if (cpu.frente == NULL) cpu.final = NULL;`
si queda vacía, final también (sin puntero colgante)

`delete aux;`
se libera la tarea ejecutada

EJECUCIÓN · COLA

La cola funcionando

```
Seleccione una opcion: 3

========================================
   PLANIFICADOR DE CPU (COLA FIFO)
========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 1

Ingrese ID de la tarea CPU: 2
Ingrese ID del proceso asociado: 1000
Ingrese descripcion de la tarea: MIL

Tarea registrada en la cola del Planificador de CPU (Encolar).

========================================
   PLANIFICADOR DE CPU (COLA FIFO)
========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 1

Ingrese ID de la tarea CPU: 4
Ingrese ID del proceso asociado: 2000
Ingrese descripcion de la tarea: DOS MIL

Tarea registrada en la cola del Planificador de CPU (Encolar).

========================================
   PLANIFICADOR DE CPU (COLA FIFO)
========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 3

========= PLANIFICADOR DE CPU (COLA FIFO) =========
ID Tarea: 2 | ID Proceso: 1000 | Descripcion: MIL <- [SIGUIENTE EN CPU]
ID Tarea: 4 | ID Proceso: 2000 | Descripcion: DOS MIL
Total de tareas pendientes: 2
```

Captura: encolar y mostrar la cola

```
========================================
   PLANIFICADOR DE CPU (COLA FIFO)
========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 2

Tarea ejecutada y despachada por el procesador:
ID Tarea: 1 | ID Proceso: 1 | Descripcion: Informe

========================================
   PLANIFICADOR DE CPU (COLA FIFO)
========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 2

No hay tareas pendientes en la CPU.
```

Captura: ejecutar hasta vaciar

Encolar

Entran las tareas 2 y 4. La tarea 2 llegó primero y queda marcada [SIGUIENTE EN CPU].

Ejecutar

Sale la tarea del frente, la primera que llegó.

Cola vacía

Sin tareas pendientes, el sistema avisa en lugar de fallar.

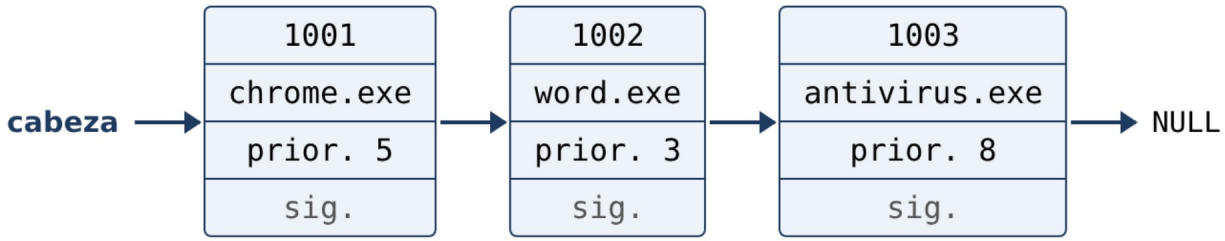
COMPARACIÓN

LIFO y FIFO lado a lado

	Pila (LIFO)	Cola (FIFO)
Idea	El último en entrar es el primero en salir	El primero en entrar es el primero en salir
Punteros	cima	frente y final
Entra por	la cima (push)	el final (encolar)
Sale por	la cima (pop)	el frente (ejecutar)
En el proyecto	Memoria RAM: bloques 101, 102 y 103	Planificador de CPU: tareas 10, 11 y 12
Enlazar / retirar	O(1)	O(1)

CÓDIGO · LISTA ENLAZADA

Lista enlazada: buscar e insertar



BUSCAR buscarProceso() · líneas 201–210

```
201 NodoLista* buscarProceso(NodoLista* cabeza, int idBuscado) {
202     NodoLista* actual = cabeza;
203     while (actual != NULL) {
204         if (actual->dato.idProceso == idBuscado) {
205             return actual;
206         }
207         actual = actual->siguiente;
208     }
209     return NULL;
210 }
```

INSERTAR AL FINAL líneas 231–243

```
231     NodoLista* nuevo = new NodoLista;
232     nuevo->dato = nuevoDato;
233     nuevo->siguiente = NULL;
234
235     if (cabeza == NULL) {
236         cabeza = nuevo;
237     } else {
238         NodoLista* actual = cabeza;
239         while (actual->siguiente != NULL) {
240             actual = actual->siguiente;
241         }
242         actual->siguiente = nuevo;
243     }
```

Las dos operaciones recorren la lista desde la cabeza con un puntero **actual** hasta llegar a NULL, por eso son **O(n)**.

EJECUCIÓN · LISTA

La lista funcionando

```
ID Proceso: 4
Nombre    : dev.exe
Prioridad : 8
-----------------------------------------
Total de procesos: 5

=========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
=========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 3

Ingrese ID de proceso a buscar: 3

Proceso encontrado -> Nombre: antivirus | Prioridad: 6

=========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
=========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 4

Ingrese ID del proceso a modificar: 3
Proceso: antivirus
Prioridad actual: 6
Ingrese nueva prioridad: 9

Prioridad actualizada correctamente.
```

Captura: buscar y modificar prioridad

```
=========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
=========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 5

Procesos ordenados por prioridad (mayor a menor).

=========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
=========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 2

========== GESTOR DE PROCESOS (LISTA ENLAZADA) ==========
ID Proceso: 15
Nombre    : 55
Prioridad : 664
-----------------------------------------
ID Proceso: 3
Nombre    : antivirus
Prioridad : 9
-----------------------------------------
ID Proceso: 4
Nombre    : dev.exe
Prioridad : 8
-----------------------------------------
ID Proceso: 2
Nombre    : excel.exe
Prioridad : 5
-----------------------------------------
ID Proceso: 1
Nombre    : chrome.exe
Prioridad : 4
-----------------------------------------
Total de procesos: 5
```

Captura: ordenar por prioridad

`Buscar`

Búsqueda secuencial por ID: encuentra a antivirus con prioridad 6.

`Modificar`

Cambiamos la prioridad del proceso 3 de 6 a 9.

`Ordenar`

Burbuja de mayor a menor. Se intercambian los datos de los nodos, no los enlaces.

CÓDIGO · VALIDACIÓN

Que el programa no se caiga

```
7   // ================ VALIDACIÓN Y LECTURA DE DATOS ================
8
9   bool entradaValida() {
10      if (cin.fail()) {
11          cin.clear();
12          cin.ignore(numeric_limits<streamsize>::max(), '\n');
13          return false;
14      }
15      return true;
16  }
17
18  bool leerEntero(const char mensaje[], int &valor) {
19      cout << mensaje;
20      cin >> valor;
21      if (!entradaValida()) {
22          return false;
23      }
24      cin.ignore(numeric_limits<streamsize>::max(), '\n');
25      return true;
26  }
27
28  bool leerTexto(const char mensaje[], char destino[], int tamano) {
29      cout << mensaje;
30      cin.getline(destino, tamano);
31      if (cin.fail()) {
32          cin.clear();
33          cin.ignore(numeric_limits<streamsize>::max(), '\n');
34          cout << "(Texto muy largo: se guardaron los primeros "
35               << tamano - 1 << " caracteres)\n";
36      }
```

Código: validación y lectura (líneas 7–36)

```
D:\UNIVERSIDAD CONTINENT

========================================
  SISTEMA DE GESTION DE PROCESOS - SO
========================================
1. Gestor de Procesos (Lista Enlazada)
2. Gestor de Memoria RAM (Pila LIFO)
3. Planificador de CPU (Cola FIFO)
4. Salir del programa
Seleccione una opcion: 1

========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 1

Ingrese ID del proceso: abc

ID invalido. Debe ser un numero mayor que 0.

========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 1

Ingrese ID del proceso: 1

Ya existe un proceso con ese ID.
```

Captura: ID no numérico y repetido

`entradaValida()`

cin.clear() y cin.ignore() evitan ciclos infinitos con letras.

`leerTexto()`

Controla textos vacíos o más largos que el arreglo.

`Antes de new`

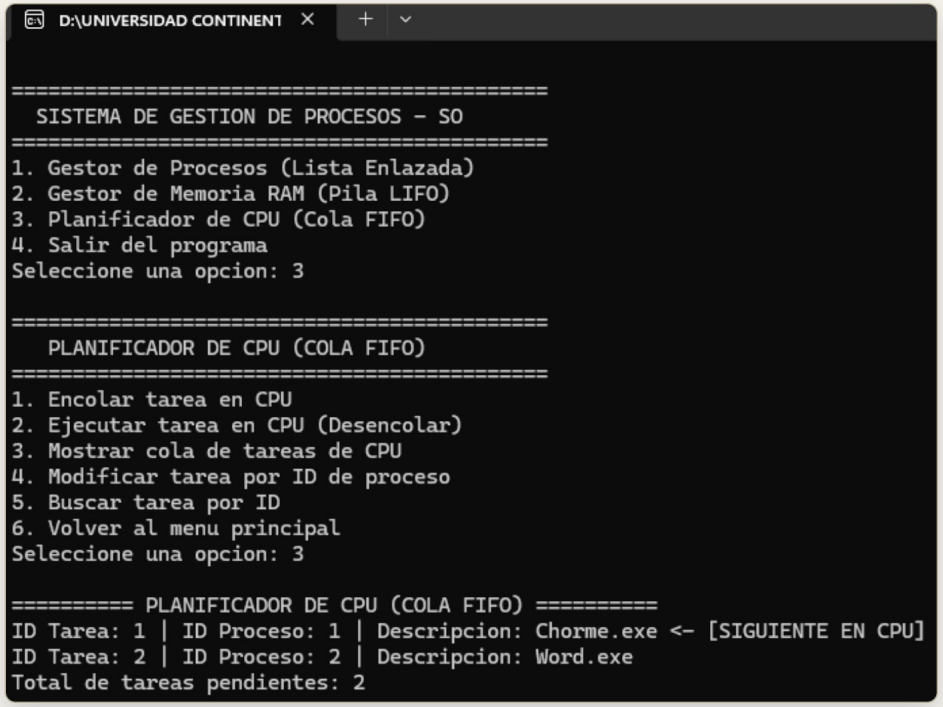
Los datos se validan en variables locales y se busca el ID repetido.

CÓDIGO · ARCHIVOS

Persistencia con fstream

```
176 void cargarTareas(PlanificadorCPU& cpu) {
177     ifstream archivo("tareas_cpu.txt");
178     if (!archivo.is_open()) return;
179     TareaCPU t;
180     while (archivo >> t.idTarea) {
181         archivo >> t.idProceso;
182         archivo.ignore(numeric_limits<streamsize>::max(), '\n');
183         archivo.getline(t.descripcion, 50);
184 
185         NodoCola* nuevo = new NodoCola;
186         nuevo->dato = t;
187         nuevo->siguiente = NULL;
188 
189         if (cpu.frente == NULL) {
190             cpu.frente = nuevo;
191         } else {
192             cpu.final->siguiente = nuevo;
193         }
194         cpu.final = nuevo;
195     }
196     archivo.close();
197 }
```

Código: cargarTareas() lee tareas_cpu.txt (líneas 176–197)



Captura: al reiniciar, la cola sigue ahí

Cada módulo guarda en su archivo con **<fstream>** después de cada cambio y lo vuelve a cargar al iniciar. Los nodos se enlazan al final para conservar el orden original.

ANÁLISIS

Complejidad de las operaciones

Operación	Tiempo	Por qué
Pop (liberar bloque)	O(1)	Solo se mueve el puntero cima
Ejecutar tarea	O(1)	Se retira el nodo del frente
Push / encolar	O(n)*	Enlazar es O(1); validar IDs repetidos recorre la estructura
Buscar proceso, bloque o tarea	O(n)	Recorre con un puntero hasta encontrar el ID
Registrar proceso	O(n)	Recorre hasta el último nodo para enlazar
Ordenar por prioridad	$O(n^2)$	Burbuja; en el mejor caso, O(n)

* Sin la validación de IDs repetidos serían O(1). Aceptamos ese recorrido extra para cuidar la integridad de los datos.

APRENDIZAJES

Lo que nos costó

Dificultad	Cómo lo resolvimos
Mezclar cin >> con getline: el salto de línea pendiente se saltaba la lectura	Agregamos cin.ignore(...) dentro de leerEntero()
Letras en campos numéricos: ciclos infinitos en los menús	entradaValida() limpia el error con cin.clear() y descarta la entrada
Puntero final colgante al vaciar la cola	Al ejecutar la última tarea, cpu.final = NULL
IDs repetidos en las tres estructuras	Se busca el ID antes de insertar (buscarProceso, buscarBloqueMemoria, buscarTarea)

CIERRE

Conclusiones

Lo que logramos

- Lista enlazada, pila LIFO y cola FIFO implementadas desde cero.
- El último bloque asignado (103) fue el primero en liberarse.
- La primera tarea encolada (10) fue la primera en ejecutarse.
- Validaciones, archivos y delete al salir: sin bloqueos ni fugas.

Lo que mejoraríamos

- Una cola de prioridad en el planificador.
- Eliminar un proceso individual de la lista.
- Validar que el ID de proceso de una tarea exista.
- Puntero al último nodo y un ordenamiento más eficiente.

Gracias

¿Alguna pregunta?

Rafael Zacarias, Jhovany Nico · Porras Benito, Flavio Isac
Buendia de la Rosa, Eric Daniel · Mayor Marticorena, Leonel Eduardo

`github.com/74171175-dot/SISTEMA-DE-GESTI-N-DE-PROCESOS-DE-UN-SISTEMA-OPERATIVO-EN-C-`